Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3676536.3676682

RESEARCH-ARTICLE

Multi-Objective Software-Hardware Co-Optimization for HD-PIM via Noise-Aware Bayesian Optimization

CHIEN YI YANG, University of California, San Diego, San Diego, CA, United States

MINXUAN ZHOU, Illinois Institute of Technology, Chicago, IL, United States

FLAVIO PONZINA, University of California, San Diego, San Diego, CA, United States

SURAJ SATHYA PRAKASH, University of California, San Diego, San Diego, CA, United States

RAID ZUHAIR AYOUB, Intel Labs, Hillsboro, OR, United States

PIETRO MERCATI, Intel Labs, Hillsboro, OR, United States

View all

Open Access Support provided by:

Illinois Institute of Technology

Intel Labs

University of California, San Diego

PDF Download
3676536.3676682.pdf
21 January 2026
Total Citations: 0
Total Downloads: 733

Published: 27 October 2024

Citation in BibTeX format

ICCAD '24: 43rd IEEE/ACM International Conference on Computer-Aided Design
October 27 - 31, 2024
NY, New York, USA

Conference Sponsors:
SIGDA

ICCAD '24: Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design (October 2024)
https://doi.org/10.1145/3676536.3676682
ISBN: 9798400710773

Multi-Objective Software-Hardware Co-Optimization for HD-PIM via Noise-Aware Bayesian Optimization

Chien-Yi Yang*, Minxuan Zhou†*, Flavio Ponzina*, Suraj Sathya Prakash*, Raid Ayoub§, Pietro Mercati§, Mahesh Subedar§, Tajana Rosing*
*University of California San Diego, †Illinois Institute of Technology, §Intel Labs
{chy036,fponzina,suraj.s2306,tajana}@ucsd.edu,mzhou26@iit.edu
{raid.ayoub,pietro.mercati,mahesh.subedar}@intel.com

ABSTRACT

In hardware accelerator design, software-hardware co-optimization requires intricate trade-offs and tight integration between software algorithms and hardware design to optimize performance, power efficiency, and area (PPA) while ensuring high accuracy. Furthermore, the inherent non-ideality in some emerging hardware technologies poses extra challenges to the co-optimization problem. This paper proposes a novel software-hardware co-optimization framework for hyperdimensional (HD) computing accelerators with emerging ReRAM-based processing in-memory (PIM) technologies, which have shown superior performance and energy efficiency over conventional machine learning accelerators. We first comprehensively characterize the non-trivial trade-offs between design parameters in HD-PIM and PPA and accuracy metrics in HD-PIM. Then, we develop a multi-objective noise-aware Bayesian optimization algorithm to find the Pareto set (optimal trade-offs between metrics) of the HD-PIM design. Our methodology uniquely addresses the stochastic nature of ReRAM by integrating error characteristics into the optimization process, thereby enhancing the quality of the generated designs. Experimental results show that our configurations achieve up to 4.28% accuracy improvement, 35.38% power reduction, 49x timing improvement, and 10% area reduction over a non-optimized design.

KEYWORDS

Multi-Objective, Bayesian Optimization, Hyperdimensional Computing, Processing-in-Memory, Resistive RAM

ACM Reference Format:
Chien-Yi Yang*, Minxuan Zhou†*, Flavio Ponzina*, Suraj Sathya Prakash*, Raid Ayoub§, Pietro Mercati§, Mahesh Subedar§, Tajana Rosing*. 2024. Multi-Objective Software-Hardware Co-Optimization for HD-PIM via Noise-Aware Bayesian Optimization. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD '24), October 27–31, 2024, New York, NY, USA.* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3676536.3676682

1 INTRODUCTION

The high accuracy of modern deep neural networks comes at the cost of extremely high computational and memory requirements. It is unfeasible to deploy such deep learning models on devices with tight power/energy budgets and limited hardware resources. As intelligent applications become more pervasive, there is a growing need for efficient machine-learning approaches that enables deployment on resource-constrained devices. Hyperdimensional (HD) computing, a promising brain-inspired computing paradigm, can deliver accurate machine learning capabilities at low computational cost and memory footprint [13–15, 22]. Unlike deep neural networks with high-dimensional parameters, HD computing represents data using high-dimensional binary vectors called "hypervectors" and performs all learning through simple vector operations, offering orders of magnitude of compactness and also robustness to noise and variations. Furthermore, HD is well-suited for hardware acceleration, especially the emerging processing in-memory (PIM) technologies that support ultra-efficient analog computing for key HD operations (e.g., similarity searches, vector multiplications, etc.) [5, 12, 21, 25, 36]. PIM architectures leverage the parallelism of memory arrays to perform computations directly where the data resides, significantly reducing energy consumption and latency.

Prior work has demonstrated the efficacy of HD computing by exploring its acceleration using different PIM architectures. However, existing studies often rely on heuristics for algorithm and hardware design, which overlooks potential enhancements and fails to explore specific trade-offs tailored to particular applications. The optimization of HD computing in the context of PIM architectures (HD-PIM) presents unique challenges, including the need to balance multiple objectives. These objectives encompass accuracy, power, performance, and area (PPA) metrics, which are interdependent and influenced by various hardware parameters such as memory size, supply voltages, operating frequencies, and the number of PIM units and software parameters, including encoding type and HD dimensionality. Notably, the inherent non-ideality in emerging PIM-enabled memory devices (e.g., resistive RAM or ReRAM) can degrade application accuracy if not properly accounted for. Moreover, the hardware non-ideality results in imprecise modeling through the optimization process, and thus downgrades the quality of optimization results, posing extra challenges to the software-hardware co-optimization problem.

In this paper, we propose a comprehensive multi-objective software-hardware co-optimization methodology to maximize the computational efficiency of HD accelerators using PIM-based ReRAM architectures while maintaining high application accuracy. Specifically, the co-optimization takes all software and hardware configurations and ReRAM noise into consideration, resulting in designs that provide better PPA and accuracy than prior non-optimized HD-PIM architectures. Also, we model four types of ReRAM noise, including

This work is licensed under a Creative Commons Attribution International 4.0 License.
ICCAD '24, October 27–31, 2024, New York, NY, USA
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1077-3/24/10.
https://doi.org/10.1145/3676536.3676682

programming noise, thermal noise, shot noise, and random telegraph noise, in PIM computation as the hardware non-ideality and optimize the HD-PIM design in consideration of the non-ideality. To demonstrate the effectiveness of our optimization framework, we optimize an HD-PIM accelerator for classification with encoding and inference, as it possesses the two fundamental aspects of the HD-PIM optimization problem: 1) PIM-induced hardware non-ideality and 2) multiple objectives, including non-trivial black-box objective functions such as accuracy. Our key contributions are:

- A comprehensive characterization of an HD-PIM classification accelerator, including the first investigation of noise impact on different HD-PIM components and trade-offs between accuracy and PPA metrics on key design parameters.
- A multi-objective software-hardware co-optimization framework via noise-aware Bayesian optimization for searching high-quality Pareto set in terms of accuracy and PPA metrics under the effect of noise.
- Experimental results show various design trade-offs with up to 4.28% accuracy improvement, 35.38% power reduction, 49x timing improvement, and 10% area reduction, compared to existing optimization algorithms and manually-designed HD-PIM accelerators.

2 RELATED WORKS

2.1 PIM-based HD accelerators

The literature is rich in applying noisy PIM on HD due to HD's noise-resistance nature [10, 30]. Karunaratne et al. [11] proposed an overall framework for in-memory hyperdimensional computing. HDnn-PIM [5] combined neural network and in-memory HD for better efficiency and accuracy. Hydrea [21] developed a robust HD system by considering wireless noise. PAAP [20] also utilized HD-PIM in their design. MicroHD [26] provides solutions for HD optimization without hardware (PIM) considerations. Nevertheless, all these works failed to include an overall software-hardware co-optimization framework to find better settings for their design in return for better accuracy and PPA metrics.

Also, despite the claim of HD's robustness against noise, none of them evaluated their design's effectiveness under real-world PIM noise. HyperMetric [32] introduced a noise-aware training method addressing ReRAM noise, but only considered the programming error. Furthermore, all of the above works only applies PIM for hypervector inference without any exploration of the application of PIM in encoding. In contrast, we will evaluate all four noise effects, programming noise, thermal noise, shot noise, and random telegraph noise, in ReRAM PIM and characterize the trade-offs of in-memory encoding by an ablation study.

2.2 Software-hardware co-optimization for DNN accelerators.

Considering both the software effectiveness and the hardware efficiency in the software-hardware co-optimization problem is a complex process. Yang et al. [33] proposed a multi-objective optimization framework for DNN mapping on ReRAM devices. However, they only considered hardware operating parameters without the consideration of architecture design and the noisiness of the accuracy function. Another work [34] developed a software-hardware co-optimization method for PIM-based neural network applications, but in an ad-hoc manner, inapplicable to HD-PIM. DOSA [9] is also a software-hardware co-optimization framework for DNN and hardware mapping and achieves better performance by exploiting the problem structure in DNN. Nevertheless, they did not take hardware area and energy consumption into consideration.

Moreover, the hardware non-ideality from ReRAM adds more complexity to HD-PIM design optimization by making precise modeling difficult. UNICO [28] used Bayesian Optimization as a design space exploration (DSE) method for software-hardware co-design with a deterministic setting. Das et al. [3] proposed a multi-objective DNN-mpping on chiplet-based accelerators. None of the above works consider the impact of hardware non-ideality in the optimization process. In this work, we propose a multi-objective noise-aware optimization framework that can deal with noisy objectives in consideration of the ReRAM noise in HD-PIM, and obtain high-quality Pareto sets for software-hardware co-design.

3 BACKGROUND

3.1 Hyperdimensional Computing

Hyperdimensional (HD) Computing is a brain-inspired lightweight machine learning method. The key idea is to represent information as hypervectors, which are often, but not limited to, in the form of $\mathbf{h} \in \{1, 0\}^d$ or $\mathbf{h} \in \{1, -1\}^d$, where $\mathbf{h}$ denotes a hypervector and d denotes the dimensionality of the hypervector. A standard hyperdimensional computing classification classifies input data through encoding and inference.

3.1.1 Encoding and Inference. A typical HD encoding is random projection (RP). Given an b-dimensional input feature vector $\mathbf{f} \in \mathbb{R}^b$, we can project the input vector into a hyperdimensional space $\{1, -1\}^b$ with a RP matrix $\mathbf{R} \in \{1, -1\}^{b \times d}$, $\mathbf{h} = \text{sign}(\mathbf{R} \cdot \mathbf{f})$. Then to classify a hypervector $\mathbf{h}$, we compare $\mathbf{h}$ with class hypervectors $\mathbf{c}_n$, where $\mathbf{c}_n$ denotes the hypervector of the nth class, by a similarity score or a distance score. Then we classify h as the class with the highest cosine similarity score, which is $\arg\max_n \text{sim}(\mathbf{h}, \mathbf{c}_n)$.

3.1.2 Training and Retraining. To train $\mathbf{c}_n$, we can aggregate all hypervectors $\mathbf{h}_i^n$ belonging to class n in the training data, which is $\mathbf{c}_n = \text{sign}(\sum_i \mathbf{h}_i^n)$. Bundling all the class hypervectors forms a set of class hypervectors for input hypervectors to compare with and make classification.

Then, to further improve the class hypervectors, we update the class hypervectors by iteratively making wrong predictions more distinguishable. In an iteration, if an input hypervector $\mathbf{h}_n$ is incorrectly classified as class m, then we update $\mathbf{c}_n$ and $\mathbf{c}_m$ by

$$\begin{cases} \mathbf{c}_n + = \alpha \cdot \mathbf{h}_n, \\ \mathbf{c}_m - = \alpha \cdot \mathbf{h}_n, \end{cases} \tag{1}$$

where α denotes the learning rate. In this work, we update the class hypervectors by $\mathbf{c}_n = \text{sign}(\mathbf{c}_n)$ to keep $\mathbf{c}_n \in \{1, -1\}^d$ at the end of the retraining epochs.

3.2 Hyperdimensional Computing with Processing-in-Memory (HD-PIM)

A common similarity score for hypervectors $\in \{1, -1\}^d$ is the cosine similarity based on dot product. This operation is highly parallelizable and extremely suited for PIM acceleration. Specifically, we focus on accelerating HD using Resistive RAM (ReRAM) PIM.

A ReRAM cell is a resistor with programmable resistance value. For a cell with conductance value G_{ij}, a local multiplication is an input voltage V_i through wordline i, and the current will be $V_i \times G_{ij}$. Accumulating the current on bitline j will get

$$I_j = \sum_i V_i \times G_{ij}. \tag{2}$$

Then ADC converts the current and returns the dot product result.

For a signed number, we can represent the number of a pair of ReRAM cells, G^+_{ij} and G^-_{ij}. Following equation 2 and subtracting the two values by $I^+_j - I^-_j$ will get the dot product value of the elements.

The encoder in HD can also use ReRAM as a dot product engine. However, there exists non-ideality in ReRAM-based PIM computation. In this work, we will assess both the impact of non-ideality using ReRAM for HD inference and for encoding. Below, we introduce the non-ideality, ReRAM noise, we take into consideration.

3.3 Resistive RAM (ReRAM) Noise

3.3.1 Programming Noise. After programming the ReRAM cells to their resistance values, the values will change following a Gaussian distribution with mean being the original resistance values. In this work, we adopt the experimental results from [19] for programming error to set the variance factor.

3.3.2 Thermal Noise and Shot Noise. Thermal noise arises from the thermal agitation of charge carriers in conductors. On the other hand, shot noise is a type of noise independent of temperature due to the discrete nature of charge transmission [16]. These two types of noise can be modeled as a Gaussian distribution together with the variance being the root-mean-square (rms) current,

$$I_{rms} = \sqrt{Gf(4K_BT + 2eV)}, \tag{3}$$

where G is the ReRAM conductance, f is the frequency, K_B is Boltzmann constant, T is the temperature, e is the elementary charge, and V is the voltage.

3.3.3 Random Telegraph Noise. Random Telegraph Noise (RTN) occurs in semiconductor devices like MOSFETs due to charge carriers getting trapped and de-trapped at defect sites within the gate oxide. We model RTN as Poisson distribution [27] with parameters reported in [8].

3.4 Multi-Objective Optimization

For HD-PIM software-hardware co-optimization, we need to formulate a multi-objective optimization problem with multiple objective functions so that we can apply optimization algorithms to the optimization problem. Suppose that we want to maximize a set of objective functions $\{f_1(\mathbf{x}), f_2(\mathbf{x}), ..., f_n(\mathbf{x})\}$ in domain X, where $\mathbf{x} \in X$, a point $\mathbf{a} \in X$ dominates another point $\mathbf{b} \in X$ iff

$$(\exists i, f_i(\mathbf{a}) > f_i(\mathbf{b})) \wedge (\forall i, f_i(\mathbf{a}) \geq f_i(\mathbf{b})), 0 < i \leq n, i \in \mathbb{N}. \tag{4}$$

Table 1: All software and hardware design parameters.

Parameter	Description
d	HD dimensionality
f_1, f_2	Kronecker product parameter. $f_1 \times f_2 = b$
d_1, d_2	Kronecker product parameter. $d_1 \times d_2 = d$
T_{Enc}	Encoding type
$freq$	Operating frequency
V_{op}	Operating voltage on ReRAM
$Temp$	Operating temperature
R_{on}, R_{off}	on/off resistance
N_{ADC}	Number of ADCs with one ReRAM
Res_{ADC}	ADC resolution
Res_{cell}	Cell resolution
S_{ReRAM}	ReRAM size
N_{PE1}, N_{PE2}	Number of PE1 and PE2 MACs in Kron. product

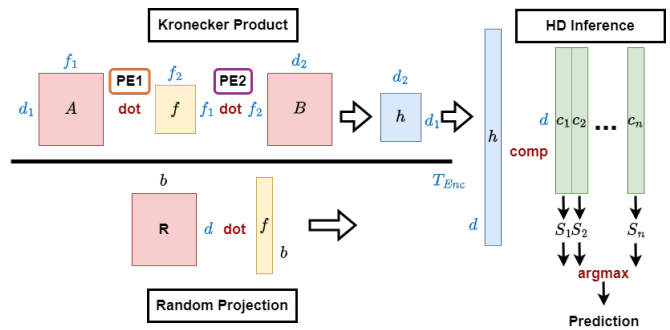

Figure 1: The HD classification algorithm and the mapping to the accelerator. Tunable design parameters are in blue. Table 1 gives a detailed description of the parameters.

On the opposite, a point $\hat{\mathbf{x}} \in X$ is non-dominated iff $\nexists \mathbf{x} \in X$, $\mathbf{x}$ dominate $\hat{\mathbf{x}}$. All non-dominated $\hat{\mathbf{x}} \in X$ form the Pareto optimal set

$$\{\hat{\mathbf{x}} | \nexists \mathbf{x} \in X, \mathbf{x} \text{ dominate } \hat{\mathbf{x}}, \hat{\mathbf{x}} \in X\}, \tag{5}$$

and the curve of the points is called Pareto frontier. A Pareto frontier represents the optimal trade-offs between all objective functions by including all the non-dominated points, which means there exists no absolutely better points than the points in the Pareto frontier.

For a subset $Y \subseteq X$, the Pareto set of Y is the set $\{\hat{\mathbf{y}} | \nexists \mathbf{y} \in Y, \hat{\mathbf{y}} \text{ dominate } \hat{\mathbf{y}}, \hat{\mathbf{y}} \in Y\}$. Our multi-objective optimization problem aims to find the closest Pareto set of Y to the Pareto optimal set, namely the optimal trade-off, for all metrics. In this work, the software-hardware co-optimization problem focuses on the Pareto set in terms of four metrics: accuracy, power, timing, and area.

4 OVERVIEW

In this work, we propose a multi-objective noise-aware optimization framework for HD-PIM, optimizing the accuracy and PPA metrics in consideration of the PIM noise, which has not been considered in previous works. The multi-objective optimization framework allows users to find the optimal trade-offs between all design metrics within reasonable time for tailored usage instead of costly design space exploration or an inflexible single design point.

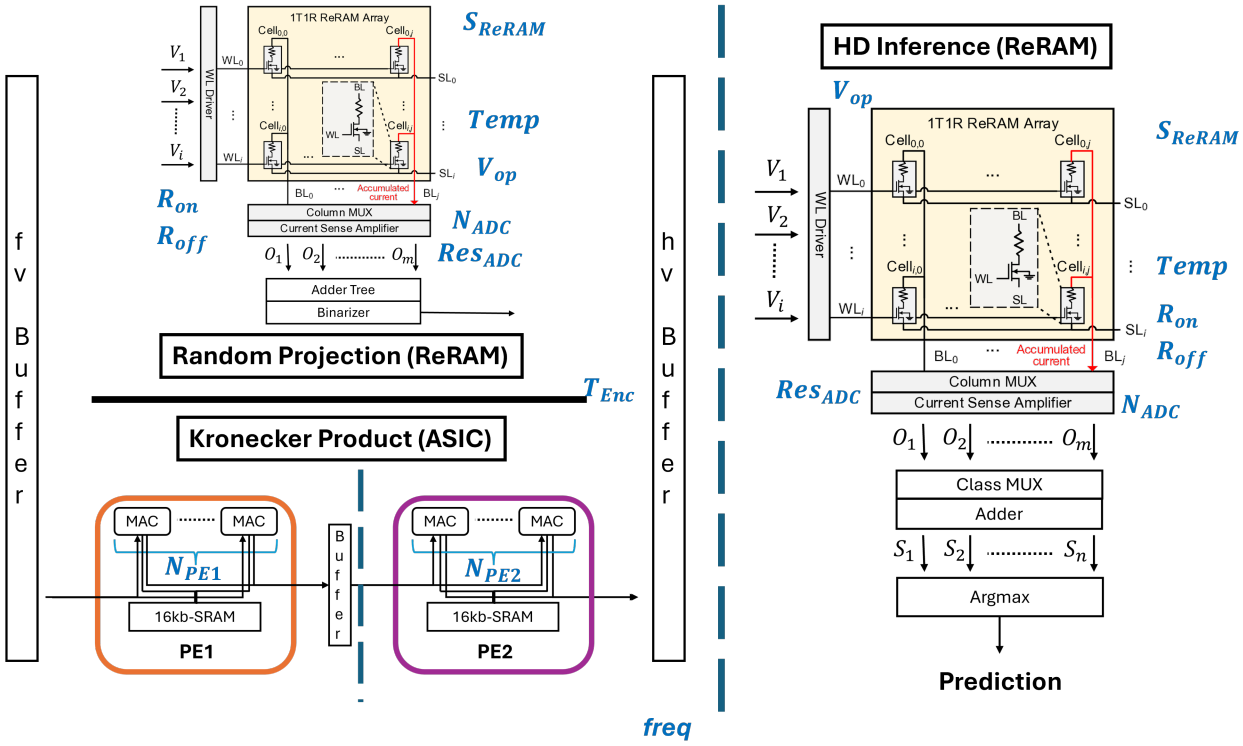

Figure 2: The HD-PIM accelerator architecture for classification. Tunable design parameters are in blue. Dash lines separate different stages in the pipeline. Table 1 describes the parameters.

To understand the HD-PIM optimization problem, we build an accuracy and PPA evaluator based on an HD classification accelerator design as illustrated in Fig. 1 and Fig. 2. The evaluator evaluates the accuracy by training an HD model of the corresponding parameters and testing the accuracy of the trained HD model under the impact of PIM noise on test workloads. Also, the evaluator simulates the PPA of the HD-PIM accelerator and returns the PPA values. Then, we conduct a comprehensive characterization on the accelerator based on the evaluator to understand the trade-offs of key design parameters and formulate the optimization problem accordingly.

The HD-PIM optimization problem has two main challenges: multiple objectives and noisy objective observation. To address the challenges, we utilize a multi-objective noise-aware Bayesian optimization algorithm. Specifically, we combine Gaussian process [6] with known noise-tolerance as the surrogate model and noisy expected hypervolume improvement (NEHVI) [4] as the acquisition function. NEHVI has two advantages under the HD-PIM setting: 1) NEHVI predicts parameters that result in better metrics in the Pareto set by taking PIM noise into consideration. 2) Hypervolume evaluates how close a data point is to the optimal Pareto front (or how far away from a "bad enough" reference point). This allows a general consideration of all objectives, including accuracy and PPA metrics, rather than combining all objectives into one.

Combining the evaluator and the optimization algorithm forms a multi-objective Bayesian optimization framework with noise consideration in the objectives. Fig. 3 illustrates the framework. The optimization algorithm trains on data from previous evaluations and actively selects the next set of software and hardware parameters for evaluation The evaluator then takes in these parameters, evaluates the corresponding accuracy and PPA, and feeds the results back into the optimization algorithm. The framework is a feedback loop that allows the algorithm to iteratively refine the parameters for improved design parameters. At the end of the feedback loop, we collect all explored points and find the Pareto set among the points through non dominated sorting.

5 HD-PIM ACCELERATOR CHARACTERIZATION

In this section, we characterize the software and hardware design parameters of the HD-PIM accelerator. We first introduce HD hyperparameters and the encoding method for HD algorithm design parameters. For the encoding, since random projection is timing-critical [31] using ASIC design, our accelerator design includes a variation to switch to Kronecker product encoding. We show how Kronecker product can approximate random projection and improve the encoding efficiency. Next, we introduce the HD-PIM architecture and the software mapping as illustrated in Fig. 1 and Fig. 2. Table 1 summarizes all design parameters. Given the architecture, we then evaluate the effect of PIM noise corresponding to different components on accuracy. Finally, we define the multi-objective optimization for our HD-PIM accelerator.

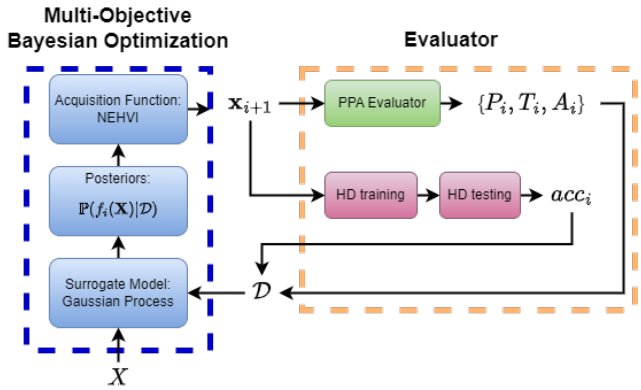

Figure 3: The evaluator evaluates the accuracy, power, performance, area of a given design parameter set and passes it to the multi-objective Bayesian optimization algorithm. Then the algorithm iteratively decides the next parameter set for the evaluator to evaluate. X is the set of parameter choices, $\mathcal{D}$ is the dataset obtained from evaluator results, f_i is one of the objective functions, and $\mathbf{x}_i$, is the next parameter set for the evaluator to evaluate. acc_i, A_i, P_i, T_i denote the parameter set, the accuracy, area, power, and timing (performance) for the ith optimization iteration.

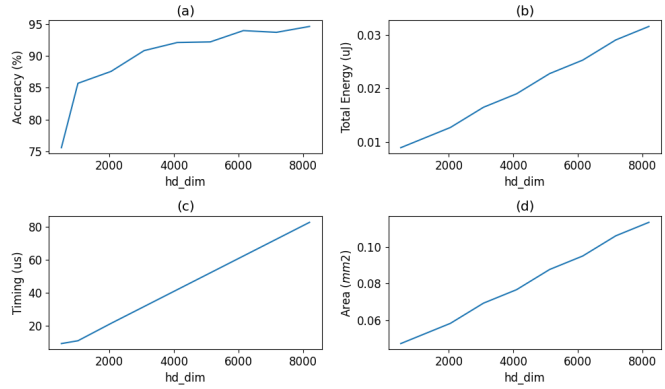

Figure 4: The (a) accuracy, (b) total energy, (c) timing, and (d) area to HD dimensionality on MNIST [17]. Default parameters are specified in Table 4.

5.1 HD Algorithm Parameters

The HD algorithm design parameters are key to the accuracy. In specific, the choice of HD dimensionality d is a key parameter to accuracy. Hypervectors with higher dimensionality represent more information, and thus results in better comparison with class hypervectors for classification. However, higher d comes at a cost of higher PPA consumption. Fig. 4 shows the accuracy and PPA trade-offs with different d choices.

Also, the conventional RP encoding requires both time complexity and memory complexity of $O(b \times d)$ and is timing-critical [31]. To make the encoding process more efficient, we can use Kronecker product (KP) [7] as an approximation to eliminate the matrix dimensionalities. We can write the approximation as

$$\mathbf{R} \cdot f \approx (\sum_{i=1}^{r} \mathbf{A}_i \otimes \mathbf{B}_i) \cdot f \approx \text{flatten}\Big(\sum_{i=1}^{r} \mathbf{A}_i \cdot \text{vec}(f) \cdot \mathbf{B}_i\Big), \quad (6)$$

where $\otimes$ denotes the KP operation, $\mathbf{A}_i \in \mathbb{R}^{d_1 \times f_1}$, $\mathbf{B}_i \in \mathbb{R}^{f_2 \times d_2}$, $f_1 \times f_2 = b$, and $d_1 \times d_2 = d$. We set $r = 1$ in this work. The time and memory complexity is now only $O(b \times d_1 + d \times f_2)$ and $O(f_1 \times d_1 + f_2 \times d_2)$ respectively. Fig. 1 illustrates the KP operation. More balanced d_1 and f_1 increase the dimensionality in encoding, and thus provide better accuracy with higher timing cost.

5.2 HD-PIM architecture Parameters

Here, we present the HD-PIM accelerator architecture as a general HD-PIM showcase and how HD works on the accelerator as illustrated in Fig. 1 and Fig. 2. The accelerator The input data first goes through the encoder with two options for the encoder, KP and RP. We implement KP with an ASIC design with two pipeline stages, processing elements 1 (PE1) and processing elements 2 (PE2). PE1 and PE2 perform the two inner product operations in KP. One group of processing elements consists of N_{PE1} and N_{PE2} MACs respectively with matrix parameters for A and B stored in a 16kb-SRAM. PE1 takes $C_{PE1} = f_2 \times \lceil \frac{f_1}{N_{PE1}} \rceil$ cycles to produce one row and passes the row to PE2. PE2 takes $C_PPE2 = d_2 \times \lceil \frac{f_2}{N_{PE2}} \rceil$ to produce one row in $h \in \mathbb{R}^{d_1 \times d_2}$. The total number of KP cycles is

$$C_{KP} = C_{PE1} + C_{PE2} + (d_1 - 1) \times \max(C_{PE1}, C_{PE2}). \quad (7)$$

Another encoding option is implementing RP with ReRAM to improve the efficiency. Res_{cell} is set as 1-bit here with elements in the RP being either 1 or -1. To calculate the number of cycles using a tiling [18] size of $S_{tile} = S_{ReRAM} \times N_{ADC}$, which is the maximum number of cells being processed parallely in a ReRAM block. This will results in $Tile_{row} = \lceil \frac{2b}{S_{tile}} \rceil$ tiles for one row since we need two cells to represent a number with possible positive and negative values. However, if $Tile_{row}$ exceeds the total number of tiles in one ReRAM $Tile_{ReRAM} = \frac{S_{ReRAM}}{N_{ADC}}$, the tiles will be distributed across different ReRAM blocks. Therefore, the number of cycles needed to process one row in $\mathbf{R}$ is $C_{row} = 2\min(Tile_{row}, Tile_{ReRAM})$ because we need to pass in positive and negative numbers as input voltage separately. Here we assume the Res_{ADC} is enough to decode the summation of all bitline current on a bitline in a ReRAM block.

The hardware then transitions to the HD inference stage with another ReRAM array. The inference calculates the similarities between the input hypervector and the class hypervectors. The prediction will be the class with the highest similarity score. Again Res_{cell} is set to be 1-bit with numbers in hypervectors being 1 or -1. Similarly, we can calculate the number of cycles needed in HD inference using the same way as in RP encoding.

5.3 Resistive RAM (ReRAM) Noise Characterization

While enormous previous works have studied the impact of noise on HD inference, no existing work has addressed the effect of noise in encoding with ReRAM to our best kno wledge. Considering noise in optimization is non-trivial because noise is a factor through which hardware design choices affect algorithm performance (accuracy).

Table 2: Ablation study on how ReRAM noise on different components impact accuracy. Training and testing are on MNIST [17]. Default parameters are specified in Table 4.

Encoding\Inference	Noiseless	ReRAM
Noiseless	90.42%	89.9%
ReRAM	87.59%	87.68%

Table 3: PPA metrics comparison of ASIC KP and ReRAM RP on MNIST [17]. Default parameters are specified in Table 4.

Encoding Type	Timing	Peak Power	Area
KP with ASIC	4.216 μs	1.421 W	0.053 mm^2
RP with ReRAM	0.032 μs	2.658 W	1.547 mm^2

To bridge the gap, we conduct an ablation study on the HD algorithm with RP encoding. Table 2 shows the HD accuracy of the combinations of RP with or without ReRAM noise and inference with or without ReRAM noise. On one hand, When applying ReRAM to HD inference, we can see little downgrade on prediction accuracy, showing the noise robustness of HD inference. On the other hand, applying ReRAM to encoding causes a slight accuracy downgrade, however, trades accuracy for better timing due to better parallelism in ReRAM. Also, we compare the PPA metrics of KP with ASICs and RP with ReRAM. Table 3 shows that using RP with ReRAM provides significant acceleration but also brings extra overhead in area and power due to the ADCs.

5.4 Optimization Problem Definition

Given the HD algorithm in Fig. 1, the HD accelerator in Fig. 2, and the range of the parameters in Table 4 as domain X, we aim to find a set $\{\mathbf{x}|\mathbf{x} \in X\}$ such that the set $\left\{\left(P(\mathbf{x}), T(\mathbf{x}), A(\mathbf{x}), acc(\mathbf{x})\right)|\mathbf{x} \in X\right\}$ form the Pareto optimal set for the objectives

$$\min P(\mathbf{x}), \min T(\mathbf{x}), \min A(\mathbf{x}), \max acc(\mathbf{x}), \tag{8}$$

where $P(\mathbf{x}), T(\mathbf{x}), A(\mathbf{x}), acc(\mathbf{x})$ returns the power, timing, area, and accuracy values of the corresponding $\mathbf{x}$ respectively. However, the observation of $acc(\mathbf{x})$, $\tilde{acc}(\mathbf{x})$ is noisy as $acc(\mathbf{x}) + \epsilon$, where ϵ is an extra noise. The problem formulation and the above evaluations allow us to build the Bayesian optimization algorithm and the feed back loop. Below, we introduce our optimization algorithm.

6 MULTI-OBJECTIVE NOISE-AWARE BAYESIAN OPTIMIZATION

In this section, we introduce our multi-objective noise-aware Bayesian Optimization algorithm. Bayesian optimization has two key components, the surrogate model and the acquisition function. The surrogate model is a probabilistic modeling to approximate objective functions that are expensive to evaluate based on prior evaluations. In this work, the prior evaluations are the combinations of the accuracy and PPA evaluations and their corresponding design parameter sets. Specifically, we set the standard Gaussian Process [6] as our surrogate model.

Table 4: Design parameters for design space exploration.

Parameter	Range	Default Value
V_{op}	–	3.3V
$Temp$	–	350K
R_{on}, R_{off}	–	3.03kΩ, 3.03 MΩ
N_{ADC}	–	8
Res_{ADC}	–	5-bit
Res_{cell}	–	1-bit
d	[512, 8192]	2048
S_{ReRAM}	{64, 128, 256}	64
T_{Enc}	{Kron., RP}	Kron
f_1	factors(b)	closest number to $\sqrt{b}$
d_1	factors(d)	closest number to $\sqrt{d}$
N_{PE1}, N_{PE2}	[8, 256]	128
$freq$	[10MHz, 1GHz]	100MHz

Based on the posterior distribution derived from the surrogate model, the acquisition function decides the next set of design parameters to evaluate. To address multiple objectives and noise, we adopt noisy expected hypervolume improvement (NEHVI) [4] as our acquisition function. The two strengths of NEHVI are 1) hypervolume evaluates the quality of a Pareto set by estimating its "volume" between the data point and a bad enough reference point instead of estimating the data point in a single function and 2) the noise evaluation is considered in the acquisition function.

6.1 Hypervolume

The hypervolume [35] (HV) indicator of a Pareto set $P \subseteq X$ against a reference point $\mathbf{r}$ is the m-dimensional Lebesgue measure $\mathrm{HV}(P|\mathbf{r}) = \lambda_m(\cup_{\mathbf{p}\in P}[\mathbf{r}, \mathbf{p}])$. where $\mathbf{r}, \mathbf{p} \in X$, X is m-dimensional, and $[\mathbf{r}, \mathbf{p}]$ denotes the hyper-rectangle bounded by $\mathbf{r}$ and $\mathbf{p}$. $\mathbf{r}$ can be chosen as any bad enough point.

Hypervolume is an indicator to evaluate the quality of a Pareto set. Larger hypervolume means 1) more points in the Pareto set and/or 2) larger distance between points in the Pareto set and the reference point, which means more trade-off options in the Pareto set and/or better quality for points in the Pareto set. Overall, multi-objective optimization aims to maximize the hypervolume.

6.2 Expected Hypervolume Improvement

To optimize our Pareto set, we want to find the next point that can best improve the hypervolume of our Pareto set. The hypervolume improvement (HVI) evaluates the quality of a new Pareto set P' over the original Pareto set P, which is $\mathrm{HVI}(P'|P, \mathbf{r}) = \mathrm{HV}(P' \cup P|\mathbf{r}) - \mathrm{HV}(P|\mathbf{r})$. We assume $\mathbf{r}$ is fixed and omit $\mathbf{r}$ below.

However, since we do not know the real objective functions, we can only estimate HVI through probabilistic modeling, the expected hypervolume improvement (EHVI). The EHVI of our HD-PIM objective functions $\mathbf{f}(\mathbf{x}) = \{P(\mathbf{x}), T(\mathbf{x}), A(\mathbf{x}), acc(\mathbf{x})\}$ is based on the posterior distribution and the Monte Carlo sampling of N samples. The EHVI for $\mathbf{x} \in X$ is

$$\mathrm{EHVI}(\mathbf{x}|P) = \mathbb{E}[\mathrm{HVI}(\mathbf{f}(\mathbf{x})|P] \approx \frac{1}{N}\sum_{t=1}^{N} \mathrm{HVI}(\tilde{\mathbf{f}}_t(\mathbf{x}|P)), \tag{9}$$

where $\tilde{\mathbf{f}}_t \sim \mathbb{P}(\mathbf{f}(\mathbf{x})|\mathcal{D})$. Using expected hypervolume improvement (EHVI) as the acquisition function takes the point that improves the hypervolume of the Pareto set as the next evaluation point, and thus improve the quality of the Pareto set.

6.3 Noisy Expected Hypervolume Improvement

Suppose our observations of function values $\mathbf{z}_i$ are noisy with $\mathbf{z}_i = \mathbf{f}(\mathbf{x}_i) + \epsilon_i, \epsilon_i \sim \mathcal{N}(0, \Sigma_i)$, where Σ_i is the noise covariance. Then we have the design parameters $X_n = \{\mathbf{x}_i\}_{i=1}^n$, the observations $Y_n = \{\mathbf{y}_i\}_{i=1}^n$, the Pareto set for P_n for $\{\mathbf{f}(\mathbf{x})|\mathbf{x} \in X_n\}$ and the dataset $\mathcal{D}_n = \{\mathbf{x}_i, \mathbf{y_i}, (\Sigma_i)\}_{i=1}^n$. To approach the functions as a distribution, noisy expected hypervolume improvement (NEHVI) is defined as

$$\text{NEHVI}(\mathbf{x}) = \int \text{EHVI}(\mathbf{x}|P_n)\mathbb{P}(\mathbf{f}|D_n)d\mathbf{f}. \tag{10}$$

Since the integral is analytically intractable, NEHVI is implemented in Monte Carlo Sampling. We can sample the objective functions as $\tilde{\mathbf{f}}_t \sim \mathbb{P}(\mathbf{f}|\mathcal{D}_n)$ and the corresponding Pareto set P_t and get

$$\text{NEHVI}(\mathbf{x}) \approx \frac{1}{N}\sum_{t=1}^{N} \text{HVI}(\tilde{\mathbf{f}}_t|P_t). \tag{11}$$

We choose NEHVI as our acquisition function in the Bayesian optimization framework to account for multiple objective functions and their noisy evaluation. We implement the models in a continuous manner and perform rounding for integral values (d, etc.). The optimization framework acts as a feedback loop where the evaluator returns the accuracy and PPA evaluations to train the surrogate model and the acquisition function guides the evaluator to find the next evaluation point. Fig. 3 illustrates the mechanism of multi-objective Bayesian optimization.

7 EVALUATION

In this section, we compare the results of different optimization methods to optimize the accuracy, peak power, total timing, and area for our HD-PIM design. To show the importance of noise consideration in optimization, we first compare results between Noisy Expected Hypervolume Improvement (NEHVI) and Expected Hypervolume Improvement (EHVI). Then, we include EHVI optimization without software parameter consideration, as per the settings in Yang et al. [33]. Finally, we discuss the Pareto set we obtain from NEHVI and show the benefit through optimization compared with a non-optimized design HyperMetric [32].

7.1 Experimental Setup

The implementation involves two components: the metric evaluator and the optimization algorithm. We implement the optimization algorithm in BoTorch [2]. For each optimization, we run 10 random search steps before running the optimization algorithm with 30 trials. For hypervolume calculation, we set our reference point $\mathbf{r}$ as accuracy = 0.6, peak power = 2 W, timing = 500 μs, area = 2 mm^2. The evaluator evaluates the accuracy and PPA metrics under given dataset. We implement the evaluator in Python and run all experiments on different datasets, including MNIST [17], pamap2 [29] and ucihar [1], as the workload. We train the HD model with a training step and 10 retraining steps with learning rate being 1.0 according to the parameters and evaluate the prediction

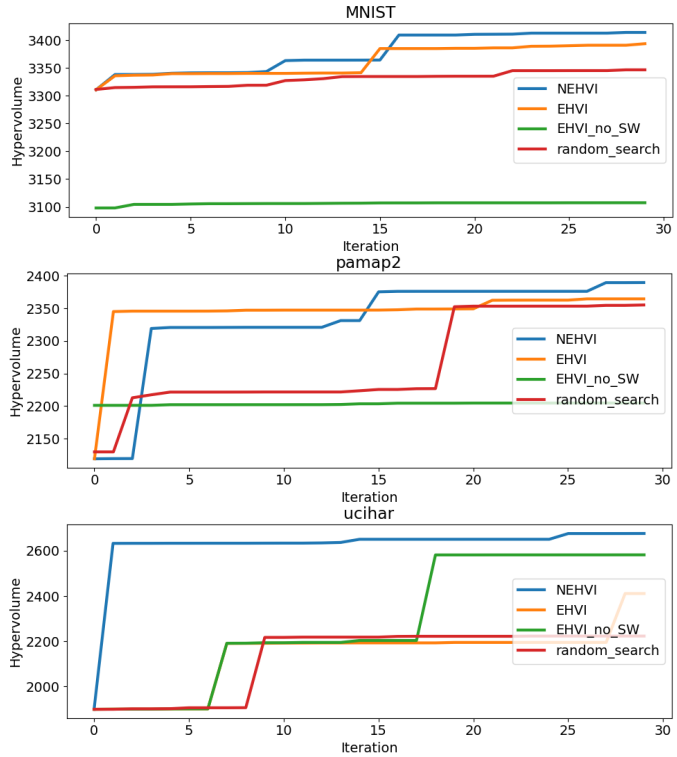

Figure 5: Hypervolume comparison between different optimization methods versus optimization iteration on different datasets, including MNIST [17], pamap2 [29] and ucihar [1].

accuracy. For ReRAM noise, we utilize PytorX [8] as the noise simulator. The accuracy evaluation is the average of 5 independent random evaluations. The PPA evaluator evaluates the peak power, the timing to run workloads on the hardware, and the hardware area. We use the method in DNN+NeuroSIM [23] adopting the 32nm technology and the parameters from [24] for PPA estimation.

Table 4 concludes the parameters and their ranges as X in our optimization experiments. We run all experiments with one 11th Gen Intel(R) Core(TM) i7-11700K CPU with 3.60GHz, 62GB memory, and an NVIDIA GeForce RTX 4090 GPU with 24GB memory.

7.2 HyperVolume Comparison

We compare the hypervolume [35] as the quality indicator of Pareto sets between methods in Fig. 5. Higher hypervolume value indicates better Pareto set quality, which means better accuracy and PPA values or more trade-off options. Overall, Bayesian optimization-guided methods perform better than random search. Noisy expected hypervolume improvement (NEHVI) is the best performing optimization method across all counterparts. Below, we discuss key aspects of the optimization problem.

7.2.1 Noise Consideration. Through the optimization process, noisy expected hypervolume improvement (NEHVI) outperforms expected hyperVolume improvement (EHVI). This is because noise undermines EHVI's ability to accurately predict better points to improve

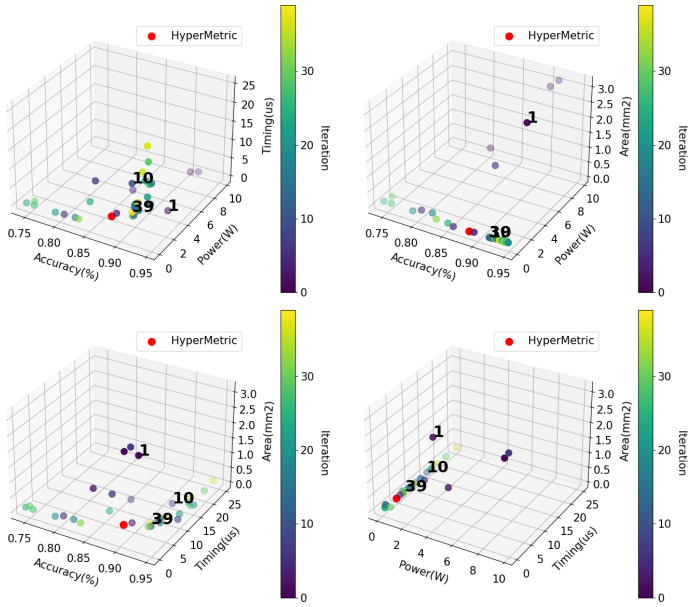

Figure 6: The Pareto set from NEHVI optimization and HyperMetric [32] in the output metric space.

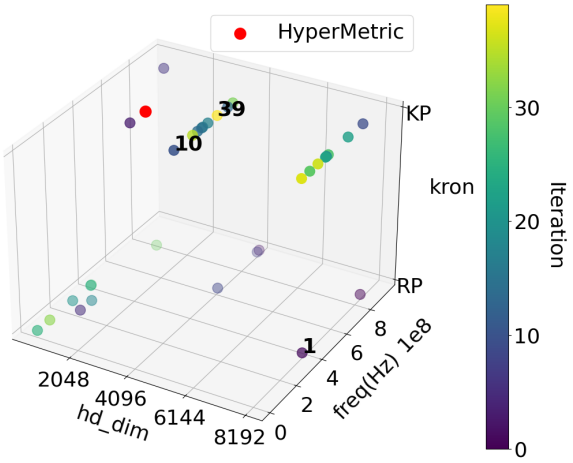

Figure 7: The Pareto set from NEHVI optimization and HyperMetric [32] in the input parameter space.

the Pareto set given the posterior, which shows the importance of noise consideration in optimization for PIM-based noisy designs.

7.2.2 Software-Hardware Optimization. Previous work [33] only optimizes hardware design without software configuration and noise consideration. In comparison, we conduct experiments using EHVI with fixed software parameters as specified in Table 4. The absence of software consideration limits the optimization space, thus significantly decreasing the performance, even worse than random search with software configuration consideration.

7.3 Pareto Set

We then demonstrate the effectiveness of software-hardware co-optimization versus a non-optimized design with MNIST as the workload. Fig. 6 shows the Pareto set obtained from NEHVI in the output metric space, the combination of four metrics in 3-D spaces, compared with the single point design HyperMetric [32]. We can find various design trade-offs in the Pareto set for HyperMetric. Newer points tend to get closer to the Pareto frontier, showing better acquisition function prediction with more evaluations fed into the surrogate model. Below we list some points with better PPA trade-offs compared with HyperMetric [32]. For power efficiency, iteration 10 gives a data point with 35.38% peak power reduction and 3.29% accuracy improvement with 3.4x delay and 4% more area. For timing, iteration 1 produces 49x timing improvement and 4.28% accuracy improvement with 66x peak power and 50x area. The main reason of the acceleration and hardware cost is the change from ASIC encoding to ReRAM-based encoding. For area, iteration 39 obtains 10% area reduction with 3.27% accuracy improvement with 1.21x peak power and 1.64x delay.

Fig. 7 shows the Pareto set in the 3-D space of three key design parameters, HD dimensionality, frequency, and encoding type. For HD dimensionality, we can see more points of 4096. This is because higher dimensionalities do not guarantee accuracy boost with higher PPA cost while low dimensionalities are generally less accurate. For the encoding choice, less RP points are in the Pareto set than KP due to the noisiness of ReRAM-based encoding and the high cost in power and area. The frequency choices for the Pareto set are more evenly distributed. The relationship between frequency, peak power, and timing demonstrates a clear trade-off: peak power is positively correlated with frequency while timing is negatively correlated. Compared with HyperMetric [32], higher HD dimensionality brings better accuracy but higher timing, RP enables better timing with more power and area consumption, and frequency provides direct trade-off between power and timing.

8 CONCLUSION

In this work, we tackle the multi-objective software-hardware co-design problem of optimizing the performance, power, area and accuracy of the HD computing paradigm with resistive RAM (ReRAM) processing-in-memory (PIM) architecture acceleration. Our novel multi-objective optimization framework co-optimizes the HD algorithm and ReRAM-PIM hardware while inherently considering ReRAM's stochastic behavior. Also, we characterized ReRAM noise impact on HD components and developed a HD classification accelerator architecture spanning algorithm/hardware parameters. Our noise-aware Bayesian optimization identifies Pareto-optimal configurations balancing accuracy and PPA metrics under realistic noise. Evaluations demonstrate up to 4.28% accuracy improvement, 35.38% power reduction, 49x timing improvement, and 10% area reduction compared with a non-optimized design.

ACKNOWLEDGMENT

We thank Weihong Xu for his precious advice. This work is supported by PRISM and CoCoSys, two research centers funded by Semiconductor Research Corporation (SRC), and National Science Foundation (NSF) funding #2112167 and #2112665.

REFERENCES

[1] Davide Anguita, Alessandro Ghio, Luca Oneto, Xavier Parra, Jorge Luis Reyes-Ortiz, et al. 2013. A public domain dataset for human activity recognition using smartphones.. In *Esann*, Vol. 3. 3.

[2] Maximilian Balandat, Brian Karrer, Daniel Jiang, Samuel Daulton, Ben Letham, Andrew G Wilson, and Eytan Bakshy. 2020. BoTorch: A framework for efficient Monte-Carlo Bayesian optimization. *Advances in neural information processing systems* 33 (2020), 21524–21538.

[3] Abhijit Das, Enrico Russo, and Maurizio Palesi. 2024. Multi-Objective Hardware-Mapping Co-Optimisation for Multi-DNN Workloads on Chiplet-based Accelerators. *IEEE Trans. Comput.* (2024), 1–14. https://doi.org/10.1109/TC.2024.3386067

[4] Samuel Daulton, Maximilian Balandat, and Eytan Bakshy. 2021. Parallel Bayesian Optimization of Multiple Noisy Objectives with Expected Hypervolume Improvement. In *Advances in Neural Information Processing Systems*, M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan (Eds.), Vol. 34. Curran Associates, Inc., 2187–2200. https://proceedings.neurips.cc/paper_files/paper/2021/file/11704817e347269b7254e744b5e22dac-Paper.pdf

[5] Arpan Dutta, Saransh Gupta, Behnam Khaleghi, Rishikanth Chandrasekaran, Weihong Xu, and Tajana Rosing. 2022. Hdnn-pim: Efficient in memory design of hyperdimensional computing with feature extraction. In *Proceedings of the Great Lakes Symposium on VLSI 2022*. 281–286.

[6] Jacob Gardner, Geoff Pleiss, Kilian Q Weinberger, David Bindel, and Andrew G Wilson. 2018. Gpytorch: Blackbox matrix-matrix gaussian process inference with gpu acceleration. *Advances in neural information processing systems* 31 (2018).

[7] Marawan Gamal Abdel Hameed, Marzieh S Tahaei, Ali Mosleh, and Vahid Partovi Nia. 2022. Convolutional neural network compression through generalized Kronecker product decomposition. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 36. 771–779.

[8] Zhezhi He, Jie Lin, Rickard Ewetz, Jiann-Shiun Yuan, and Deliang Fan. 2019. Noise Injection Adaption: End-to-End ReRAM Crossbar Non-ideal Effect Adaption for Neural Network Mapping. In *Proceedings of the 56th Annual Design Automation Conference 2019* (Las Vegas, NV, USA) *(DAC '19)*. Association for Computing Machinery, New York, NY, USA, Article 57, 6 pages. https://doi.org/10.1145/3316781.3317870

[9] Charles Hong, Qijing Huang, Grace Dinh, Mahesh Subedar, and Yakun Sophia Shao. 2023. DOSA: Differentiable Model-Based One-Loop Search for DNN Accelerators. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture* (, Toronto, ON, Canada,) *(MICRO '23)*. Association for Computing Machinery, New York, NY, USA, 209–224. https://doi.org/10.1145/3613424.3623797

[10] Pentti Kanerva. 2009. Hyperdimensional computing: An introduction to computing in distributed representation with high-dimensional random vectors. *Cognitive computation* 1 (2009), 139–159.

[11] Geethan Karunaratne, Manuel Le Gallo, Giovanni Cherubini, Luca Benini, Abbas Rahimi, and Abu Sebastian. 2020. In-memory hyperdimensional computing. *Nature Electronics* 3, 6 (2020), 327–337.

[12] Arman Kazemi, Mohammad Mehdi Sharifi, Zhuowen Zou, Michael Niemier, X Sharon Hu, and Mohsen Imani. 2021. Mimhd: Accurate and efficient hyperdimensional inference using multi-bit in-memory computing. In *2021 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)*. IEEE.

[13] Behnam Khaleghi, Jaeyoung Kang, Hanyang Xu, Justin Morris, and Tajana Rosing. 2022. GENERIC: highly efficient learning engine on edge using hyperdimensional computing. In *Proceedings of the 59th ACM/IEEE Design Automation Conference* (San Francisco, California) *(DAC '22)*. Association for Computing Machinery, New York, NY, USA, 1117–1122. https://doi.org/10.1145/3489517.3530669

[14] Behnam Khaleghi, Hanyang Xu, Justin Morris, and Tajana Šimunić Rosing. 2021. tiny-HD: Ultra-Efficient Hyperdimensional Computing Engine for IoT Applications. In *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 408–413. https://doi.org/10.23919/DATE51398.2021.9473920

[15] Yeseong Kim, Mohsen Imani, and Tajana S Rosing. 2018. Efficient human activity recognition using hyperdimensional computing. In *Proceedings of the 8th International Conference on the Internet of Things*. 1–6.

[16] LB Kish and CG Granqvist. 2000. Noise in nanotechnology. *Microelectronics Reliability* 40, 11 (2000), 1833–1837.

[17] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. [n. d.]. Gradient-based learning applied to document recognition. *Proc. IEEE* 86, 11 ([n. d.]), 2278–2324.

[18] Meng-Yao Lin, Hsiang-Yun Cheng, Wei-Ting Lin, Tzu-Hsien Yang, I-Ching Tseng, Chia-Lin Yang, Han-Wen Hu, Hung-Sheng Chang, Hsiang-Pang Li, and Meng-Fan Chang. 2018. DL-RSIM: A simulation framework to enable reliable ReRAM-based accelerators for deep learning. In *2018 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 1–8.

[19] Yu-Hsuan Lin, Chao-Hung Wang, Ming-Hsiu Lee, Dai-Ying Lee, Yu-Yu Lin, Feng-Min Lee, Hsiang-Lan Lung, Keh-Chung Wang, Tseung-Yuen Tseng, and Chih-Yuan Lu. 2019. Performance Impacts of Analog ReRAM Non-ideality on Neuromorphic Computing. *IEEE Transactions on Electron Devices* 66, 3 (2019), 1289–1295. https://doi.org/10.1109/TED.2019.2894273

[20] Fangxin Liu, Haomin Li, Ning Yang, Yichi Chen, Zongwu Wang, Tao Yang, and Li Jiang. 2024. PAAP-HD: PIM-Assisted Approximation for Efficient Hyper-Dimensional Computing. In *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 46–51.

[21] Justin Morris, Kazim Ergun, Behnam Khaleghi, Mohen Imani, Baris Aksanli, and Tajana Simunic. 2022. Hydrea: Utilizing Hyperdimensional Computing for a More Robust and Efficient Machine Learning System. *ACM Transactions on Embedded Computing Systems* 21, 6 (2022), 1–25.

[22] Justin Morris, Mohsen Imani, Samuel Bosch, Anthony Thomas, Helen Shu, and Tajana Rosing. 2019. CompHD: Efficient hyperdimensional computing using model compression. In *2019 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)*. IEEE, 1–6.

[23] Xiaochen Peng, Shanshi Huang, Yandong Luo, Xiaoyu Sun, and Shimeng Yu. 2019. DNN+ NeuroSim: An end-to-end benchmarking framework for compute-in-memory accelerators with versatile device technologies. In *2019 IEEE international electron devices meeting (IEDM)*. IEEE, 32–5.

[24] Xiaochen Peng, Rui Liu, and Shimeng Yu. 2019. Optimizing weight mapping and data flow for convolutional neural networks on processing-in-memory architectures. *IEEE Transactions on Circuits and Systems I: Regular Papers* 67, 4 (2019), 1333–1343.

[25] Prathyush Poduval, Mariam Issa, Farhad Imani, Cheng Zhuo, Xunzhao Yin, Hassan Najafi, and Mohsen Imani. 2021. Robust In-Memory Computing with Hyperdimensional Stochastic Representation. In *2021 IEEE/ACM International Symposium on Nanoscale Architectures (NANOARCH)*. IEEE, 1–6.

[26] Flavio Ponzina and Tajana Rosing. 2024. MicroHD: An Accuracy-Driven Optimization of Hyperdimensional Computing Algorithms for TinyML systems. *arXiv preprint arXiv:2404.00039* (2024).

[27] Francesco Maria Puglisi, Luca Larcher, Andrea Padovani, and Paolo Pavan. 2015. A complete statistical investigation of RTN in HfO 2-based RRAM in high resistive state. *IEEE Transactions on Electron Devices* 62, 8 (2015), 2606–2613.

[28] Bahador Rashidi, Chao Gao, Shan Lu, Zhisheng Wang, Chunhua Zhou, Di Niu, and Fengyu Sun. 2023. UNICO: Unified Hardware Software Co-Optimization for Robust Neural Network Acceleration. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 77–90.

[29] Attila Reiss and Didier Stricker. 2012. Introducing a new benchmarked dataset for activity monitoring. In *2012 16th international symposium on wearable computers*. IEEE, 108–109.

[30] Anthony Thomas, Sanjoy Dasgupta, and Tajana Rosing. 2021. A theoretical perspective on hyperdimensional computing. *Journal of Artificial Intelligence Research* 72 (2021), 215–249.

[31] Weihong Xu, Jaeyoung Kang, and Tajana Rosing. 2023. Fsl-hd: Accelerating few-shot learning on reram using hyperdimensional computing. In *2023 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1–6.

[32] Weihong Xu, Viji Swaminathan, Sumukh Pinge, Sean Fuhrman, and Tajana Rosing. 2023. HyperMetric: Robust Hyperdimensional Computing on Error-prone Memories using Metric Learning. In *2023 IEEE 41st International Conference on Computer Design (ICCD)*. 243–246. https://doi.org/10.1109/ICCD58817.2023.00045

[33] Xiaoxuan Yang, Syrine Belakaria, Biresh Kumar Joardar, Huanrui Yang, Janardhan Rao Doppa, Partha Pratim Pande, Krishnendu Chakrabarty, and Hai Helen Li. 2021. Multi-Objective Optimization of ReRAM Crossbars for Robust DNN Inferencing under Stochastic Noise. In *2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. 1–9. https://doi.org/10.1109/ICCAD51958.2021.9643444

[34] Xiaoxuan Yang, Shiyu Li, Qilin Zheng, and Yiran Chen. 2023. Improving the robustness and efficiency of PIM-based architecture by SW/HW co-design. In *Proceedings of the 28th Asia and South Pacific Design Automation Conference*. 618–623.

[35] Eckart Zitzler and Lothar Thiele. 1999. Multiobjective evolutionary algorithms: a comparative case study and the strength Pareto approach. *IEEE transactions on Evolutionary Computation* 3, 4 (1999), 257–271.

[36] Zhuowen Zou, Hanning Chen, Prathyush Poduval, Yeseong Kim, Mahdi Imani, Elaheh Sadredini, Rosario Cammarota, and Mohsen Imani. 2022. Biohd: an efficient genome sequence search platform using hyperdimensional memorization. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*. 656–669.